Super-Precise Latency: How CPU Threading Distorts Benchmarks, an Industry-Wide Audit, and How to Fix It

Gustavo Pedro Ricou and Romaric Duvignau

Abstract—Engineers choose message brokers partly by benchmarks reporting latencies below a millisecond. In our experimental settings, our producer and consumer share the same machine. The producer reads the clock after the broker's acknowledgment arrives, and the consumer after the message arrives. The reading of the consumer minus that of the producer spreads over milliseconds and sometimes falls below zero. Values below zero depend partly on whether the broker acknowledges or delivers first. But on both brokers in our settings, these values come mostly from threads that wait for a processor core. The producer's thread can wait so long that the consumer reads first. Controlled experiments tested laws we derived: values below zero become more common under processor load and rarer when clock-reading threads get real-time priority. The consumer's thread can wait too, which delays its next request. A message can then wait at the broker. Only the broker sees that wait. But an industry-wide audit finds that almost no broker reports it. Most measuring tools we audited drop or replace values below zero without counting them, or never measure them. To fix these problems, we give a general solution for tools and brokers, and test the tools' part.

Index Terms—Apache Kafka; benchmark auditing; kernel timestamping; latency benchmarking; message brokers; Redis Streams; thread scheduling; timestamp resolution.

1 Introduction

Many online services move their data through a message broker. A broker is a service that passes messages between programs (Fig. 1a). The program that publishes a message is the *producer*. The program that receives it is the *consumer*. Engineers choose among brokers partly by latency benchmarks [1]. These benchmarks time each message from the producer to the consumer. They now report latencies below a millisecond [2]. At that scale, a thread's wait for a processor core can be longer than the latency itself. So a team that ranks brokers by such a number may act on how the number was measured, and not on the broker. The same holds for a team that promises a response time with such a number. This paper shows how to separate the two and how to measure the broker alone.

The problem matters beyond any one benchmark. Delays of microseconds now shape how datacenter systems are built [3]. A large service is only as fast as the slowest of its many parts [4], so its designers need latencies that they can trust. Yet an experiment on a computer system can produce wrong data although nothing in it looks wrong [5]. Scientists in many fields share this concern [6]. Our first benchmark timed each message from the broker's acknowledgment. A latency timed that way understates the message's trip from the producer to the consumer several-fold. It also distorts comparisons between brokers (Section 3).

We study one difference. The broker writes each message to its log. Then it returns an acknowledgment to the producer and delivers the message to the consumer. Let the producer and the consumer run on one machine and read its one clock. The producer reads the acknowledgment at t_{ack}, and the consumer reads the message at t_{recv}. On a broker that acknowledges first, the two readings should come in that order. They should be apart only by a small positive margin. This margin is how much later the message reaches the consumer than the acknowledgment reaches the producer. We make four contributions.

1) **The distribution** (Section 3). On one clock, $S = t_{\text{recv}} - t_{\text{ack}}$ spreads over milliseconds, and 8.79% of its values fall below zero. Nothing arrived early. The *publish call* is the moment the producer hands the message over. No latency timed from the publish call falls below zero.
2) **The mechanism, and its laws** (Section 4). A thread must wait for a core before it can read the clock. On Kafka, the kernel's timestamps show the acknowledgment arriving first. There the thread that reads the acknowledgment can wait for a core until the consumer has read the message. Redis often delivers first, so a shorter wait is enough. But on both brokers, most values below zero come from such waits. The share of S below zero rises with processor load and depends on where the load runs. It falls on longer paths. Its timescale is the scheduler's slice, which the CPU count sets. *Real-time priority* runs the threads that read the clock ahead of every thread at the default *normal priority*. The *background load* is the processor work that runs beside the clients. Real-time priority cut that share at least 4.4-fold on Kafka and 4.2-fold on Redis. It did so in every matched pair, while the background load stayed the

- *G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.*
- *R. Duvignau is with the Department of Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden. E-mail: duvignau@chalmers.se.*

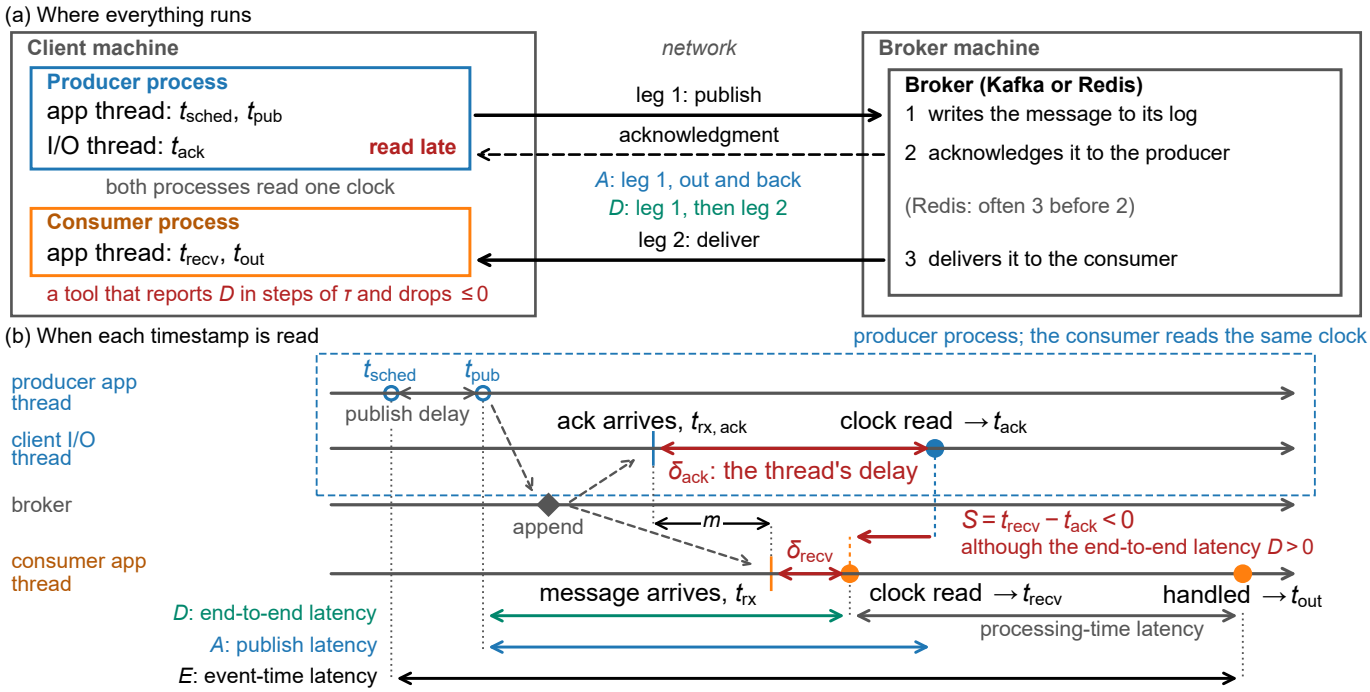


Fig. 1. **Every message crosses two legs, and a thread may wait for a core before it reads the clock.** (a) The producer and the consumer are two processes on one client machine, and they read its one clock. The broker runs on a second machine. Leg 1 carries a message to the broker, which writes it to its log and returns an acknowledgment. Leg 2 carries the message on to the consumer. (b) The producer reads t_{pub} as it publishes. The acknowledgment and the message reach the client machine at the kernel's receive timestamps $t_{\mathrm{rx,ack}}$ and t_{rx}, which lie the margin m apart (Section 4.1). The client's I/O thread reads t_{ack} only after its delay δ_{ack}, which holds any wait for a core. The consumer reads t_{recv} after its own delay δ_{recv}. Here A runs past D, so $S = D - A$ is below zero. Yet the message arrived after its acknowledgment. The red notes mark this late reading. A red note below the consumer's box marks a tool that drops every value at or below zero. That tool's clock moves in steps of τ (Section 5). E runs from t_{sched} to t_{out} (Section 2). The path drawn is Kafka's. On Redis, the application thread reads the acknowledgment. In most runs the delivery arrives first, so $m < 0$.

same. The same wait can also delay the next request of a consumer that keeps one request at a time. A message can then wait at the broker for that request.

3) **The industry-wide audit** (Section 5). We read 24 message tools at source to see what each does with a latency at or below zero. Of these, 19 cut such a latency out of a figure they report. All but one of them count none of the values they cut. The one that counts them is the Storage Benchmark Kit (SBK). Under the strictest reading of our classification, twelve still cut them. Of the 32 tools we ran, 19 take both timestamps in one process, where no clock offset can reach them. But eight of those time a round trip and four only the acknowledgment. Neither is a delivery's *one-way delay* [7]. A one-way delay is the time from the sender to the receiver. The rest cannot see the wait of a separate receiving process. On a millisecond clock, a filter that keeps only latencies above zero also deletes the fastest messages. No public report we audited says how many samples such a filter kept. The brokers that the industry's tools time report at most when a message arrived, or its whole time at the broker in milliseconds.
4) **What to do** (Section 6). Checks for benchmark authors and readers, each with what it gave where we measured it, and a general method for a tool. The method takes the kernel's timestamp of every packet on the client machine, which a tool can read on its own sockets in four steps. It matches each message to its packets by the id that the broker gives it. From those timestamps alone, a tool can check whether a request was waiting at the broker when each message arrived. The latency between the kernel's timestamps held still across background load and priority. We tested it at fifty messages a second on one pair of machines. Later tests, whose predictions we committed before their runs, ran on two pairs of machines at 2,000 messages a second. On one pair, consumers that kept requests waiting at the broker held the tail of that latency still under load on Redis and NATS. On RabbitMQ, a consumer that gave the broker a standing permission to deliver more messages did the same. On Kafka, one client has a fetch thread that does nothing else. At real-time priority, that thread kept nearly all of the consumer's own delay out of that latency. A broker can report the times that only it sees, in a few bytes per message, when a consumer asks for them.

Together, these results show that a broker latency below a millisecond describes both the broker and how it was timed. They also give the means to time the broker alone.

2 How a Message Is Timed

A producer process publishes messages to a broker at a paced rate. A consumer process reads them back. Each

message crosses two legs (Fig. 1a). Leg 1 carries it to the broker. The broker writes it to its log and returns an acknowledgment along leg 1. Leg 2 carries the message on to the consumer. Three timestamps mark its path. The first is t_{pub}, read immediately before the publish call. The second is t_{ack}, read when the producer learns that the broker has accepted the message. The third is t_{recv}, read when the consumer holds the message. Their differences give three quantities:

$$D \equiv t_{\text{recv}} - t_{\text{pub}}, \qquad A \equiv t_{\text{ack}} - t_{\text{pub}},$$
$$S \equiv t_{\text{recv}} - t_{\text{ack}} \;=\; D - A. \tag{1}$$

Broker benchmarks call D the *end-to-end latency* [1]. It is the message's time from the publish call to the consumer. They call A the *publish latency*. It is the producer's wait to hear that the broker has the message. S compares the acknowledgment's return along leg 1 with the message's trip along leg 2. On Kafka, the broker returns the acknowledgment before it answers the consumer's waiting request. Redis writes the delivery to a waiting consumer first. A benchmark that starts its clock at the acknowledgment reports S as the message's latency. Every publish latency also carries whatever delays the reading of its acknowledgment (Section 5.2). Two more readings place a message in its workload. The reading t_{sched} is when the replayed schedule made it due, and t_{out} is when the consumer had handled it. The *event-time latency* $E = t_{\text{out}} - t_{\text{sched}}$ holds D [8]. The *publish delay* $t_{\text{pub}} - t_{\text{sched}}$ comes before D, and the consumer's *processing-time latency* $t_{\text{out}} - t_{\text{recv}}$ comes after it.

The measurements come from Oracle Cloud virtual machines on a private network, unless the text says otherwise. One machine runs both brokers, Kafka and Redis Streams, each as a single node. Kafka has one partition and one replica, so `acks=all` is the leader's own append (Supplement S1.2). Another machine runs the producer and the consumer as separate CPython processes. It is the *client machine*. The processes use the kafka-python and redis-py client libraries, and Supplement S1 lists their versions where these were recorded. Both processes read one clock, so nothing here depends on synchronizing clocks. The client machine's kernel is Linux 6.8, whose scheduler is EEVDF [9]. EEVDF stands for Earliest Eligible Virtual Deadline First.

The producer replays StatsBomb's open football event data [10], and only its message sizes and rates matter here. Each message is a small JSON record of an event's identifiers and timestamps (Supplement S1.1). In nine runs out of ten, a producer publishes 0.60–1.03 messages a second, as a sparse event feed does. *Utilization* ρ is the fraction of processor time that is busy on the client machine over a run. The background load sets it, and that load runs only on the client machine.

A *condition* is one point of the experimental design. It fixes a broker, a publish rate, a message size, a background load and how often the consumer confirms what it has read. A *run* executes a condition once, and a *replicate* is a further run of it. A *campaign* is a set of runs designed together. Some experiments change one variable and hold the rest fixed. Each value that such an experiment gives the variable is a *configuration*. A later campaign ran on three pairs of Microsoft Azure machines, two x86 and one Neoverse N2. It used the same brokers and the same clients. It also used

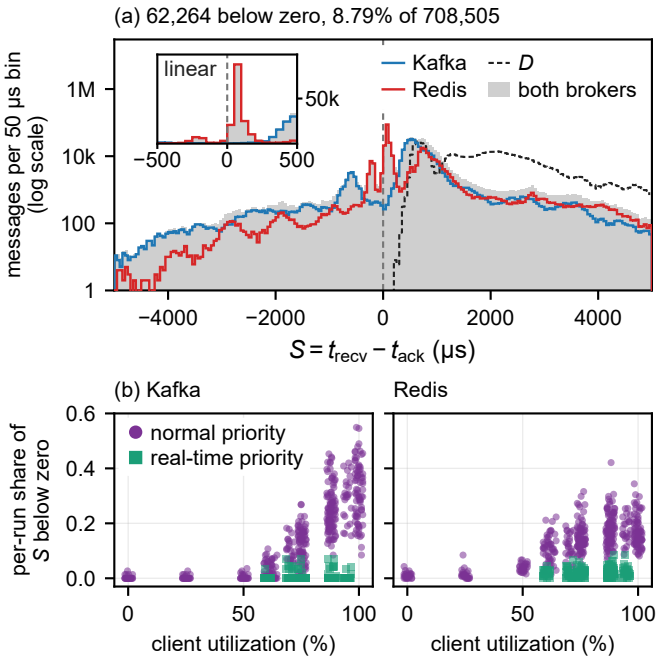


Fig. 2. S **spreads over milliseconds, and part of it falls below zero.** (a) $S = t_{\text{recv}} - t_{\text{ack}}$ for the 708,505 messages of our cloud runs, of which 62,264 are below zero. Each value is read on one clock at nanosecond resolution. The counts are in 50 μs bins over ± 5 ms, on a log scale, and on a linear scale in the inset. The black dashed line is D on the same messages, and it is never below zero. Redis's values below zero gather in a mode near -200 μs (Supplement S3.11). (b) The share of S below zero in each of 1,250 runs of the load and priority campaigns (Supplement S3), at its configuration's utilization and offset sideways. The share rises with utilization at normal priority and stays low at real-time priority.

Kafka's Java client (Supplement S1.1). It was *registered*. We committed its predictions to the project's public repository before the runs that test them [11] (Supplement S6.1). Two more registered campaigns, plan versions 47 and 49, tested the general method of Section 6.1 on two of these pairs (Supplement S8). Supplement S1.4 lists the runs behind most numbers.

3 The Distribution of S

On a broker that acknowledges first, S is ideally a small positive margin (Section 4.1). Measured, S spreads over milliseconds, and part of it falls below zero (Fig. 2). In the cloud runs that produced matched records, 8.79% of the values of S fall below zero (Fig. 2a). This proportion is the share of S below zero. It pools the runs of every load. Each run counts in proportion to its number of messages. Per run, the share varies widely, and some runs have none (Supplement S1.3). It rises with utilization and falls at real-time priority (Fig. 2b). Both brokers show it (Supplement S1.3). The registered campaign found values below zero again with Kafka's own Java client. The campaign found them on Neoverse N2 as on x86 (Supplement S6.2).

Nothing arrived early. On the same messages and the same clock, neither D nor A falls below zero once. The smallest value of each is far from zero, while S reaches -99.8 ms. Only S falls below zero, and it does on both brokers (Supplement S1.3). So the two readings came out

of order. This was mostly because a thread waited for a core (Section 4.1). A widely run broker benchmark once reported an average publish latency above its average end-to-end latency, with the producer and the consumer on one node. Its user was told that the clocks were skewed between nodes [12]. One clock has no skew to blame.

A benchmark that reads S as the latency reports a value several times smaller than D. The runs form groups, one for each condition and each verdict of the check for values below zero (Section 6.2; Supplement S2.1). At the median over a broker's groups, S understates D $3.8\times$ on Kafka and $6.6\times$ on Redis (Supplement S2.1). S also distorts the gap between the brokers. The gap is Kafka's median over Redis's. The distortion is the gap read from S over the gap read from D. We compared the brokers on matched workloads. A matched workload is one number of producers running at once on each broker, one of those producers and one verdict of the check. Both brokers have runs of it. There S widens the gap in 79% of the pairs and narrows it in the rest. The median distortion is $1.7\times$ (Supplement S2.1).

4 The Mechanism and Its Laws

A thread must wait for a core before it can read the clock. That wait is the thread's *scheduling delay*. It is the time that a thread ready to run spends queued before a core runs it. Real-time Linux measures it as scheduling latency [13]. The kernel's timestamps and *controlled experiments* show when this wait puts S below zero. A controlled experiment changes one variable and holds the rest fixed.

4.1 When S falls below zero

D and A are causal chains. Each starts at the publish call, which comes before the broker's write. The write comes before both the acknowledgment and the delivery. S is not a chain. Its two readings are two branches of one cause, the write, and neither has to come first. Equation 1 holds for every message, whatever the signs, so

$$S < 0 \quad \text{if and only if} \quad A > D. \tag{2}$$

If neither thread waited, S would equal the *margin* m. The margin is the time that the message still needs to reach the consumer once the acknowledgment has reached the producer. Each reading is late by its thread's delay, which holds any wait for a core (Fig. 1b). So S falls below zero when the acknowledging thread's delay, δ_{ack}, is longer than the margin plus the consumer's own delay, δ_{recv}. We measured the margin as $S_k = t_{\mathrm{rx}} - t_{\mathrm{rx,ack}}$. It is the difference between the kernel's receive timestamps of the message and of its acknowledgment. No thread's delay enters S_k. On Kafka, the acknowledgment came first, by a median of 72–92 μs. Under load, 99.8% of the values of S below zero had $S_k \geq 0$, so they are late readings. On Redis with sixteen publish workers, the delivery came first in 16 of its 24 runs. Redis's margin was then below zero, so a shorter wait, or none, could put S below zero. For 95–100% of its values below zero, the delivery had come first (Supplement S3.11). But in the controlled experiments, most values below zero on both brokers came from waits for a core (Section 4.2).

4.2 How often, and what moves the share

Each thread that reads the clock is either running or waiting for a core. A running thread's reading is late by a narrow jitter. On Kafka, the share of S below zero grows $61\times$ from an idle host to the heaviest load before its processors are fully busy. Over the same range, the jitter's width only grows $5\times$. So load mostly changes how often the thread is waiting. Write p for the probability that the thread is waiting for a core. Write G for the chance that the rest of its wait outlasts a given time. If the consumer's own wait is short, then to leading order

$$\Pr\big[\, S < 0 \mid m \,\big] \;\approx\; p\,G(m), \tag{3}$$

the probability that the thread is waiting, times G at the margin. This is the textbook decomposition of a wait, here read from the order of two clock readings. Load, its placement and priority move p, and the path's length moves m.

Priority. Real-time priority for the threads that read the clock should lower p while utilization stays fixed. Every controlled experiment ran both brokers (Supplement S3). Each controlled experiment moved the share on both brokers. Real-time priority cut the share 7- to 80-fold on Kafka and 6.0- to 7.8-fold on Redis, over matched pairs at the same utilization. It cut it at least 4.4- and 4.2-fold in every pair, the smallest of the pairs' run-level lower bounds (Supplement S3.1). That refutes any account in which utilization alone sets the share, including the queueing form we had ourselves adopted. In the registered campaign, on other machines, real-time priority removed nearly every value below zero on Kafka. Load raised the share in all five Kafka campaigns that tested it (Supplement S6.2).

Placement. The background load was either concentrated on six of the eight cores or spread over all of them. The two configurations reached the same utilization, and on both brokers they gave different shares (Supplement S3.2). On Kafka, the spread load gave the higher share, 2.07-fold, and 2.02-fold in a full replication. Spreading the load leaves no core free, so a woken thread waits more often. On Redis, the concentrated load gave more values below zero. So where the load runs matters on both brokers, in opposite directions.

Path length. A wait of a given size overlaps a short path almost entirely and a long one hardly at all. Padding the payload lengthens the time to the consumer's reading, by transfer and parsing, at nearly unchanged utilization. On Kafka, it raised the median of S 74-fold. The share fell 3.8-fold. On Redis, the share fell 7.9-fold. That is the direction G predicts, and the opposite of what stress on the host would cause (Supplement S3.3).

The wait, timed message by message. The registered campaign timed each acknowledgment's own wait for a core (Supplement S6.1). Of the readings below zero, 79.4–92.0% had spent most of the time from the reply's arrival to the timestamp waiting for a core. With nothing fitted, we predicted the share of S below zero from the share of acknowledgments whose own wait outlasted the run's margin. At the median, the predictions were 1.11–1.30 times the measured share. The registered prediction held on four of the six pair and broker combinations (Supplement S6.2). In the runs timed by the kernel, load raised the 99th percentile

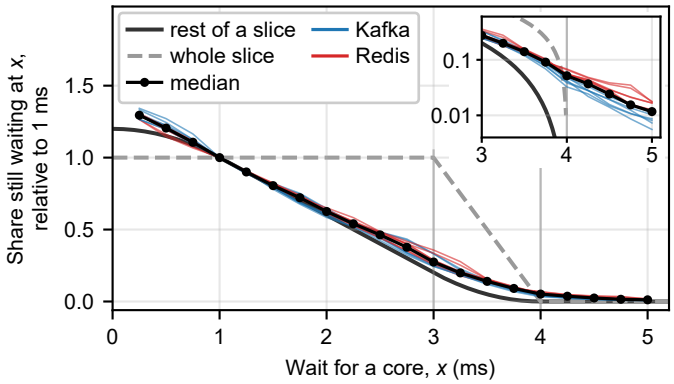


Fig. 3. **Woken by the reply, a thread waits out the rest of a slice, not a whole one.** Each thin line is one of the registered campaign's twelve combinations of machine pair, broker and load, and the bold line is their median. A line gives the share of acknowledgments whose thread was still waiting for a core at x, over the share at 1 ms, a tick. This puts every combination on one scale with nothing fitted. Of $1{,}516{,}342$ acknowledgments in 304 runs, 8.6–18.2% on Kafka and 4.8–10.9% on Redis waited past 1 ms. These runs set the slice s by hand to 3 ms and the tick h to 1 ms, and vertical lines mark s and $s + h$. The dark curve waits out the rest of a slice and then up to a tick (Supplement S6.3), and the dashed curve waits a whole slice and a tick. The inset shows the tail past 3 ms on a log scale, which the dark curve sets at zero past $s + h$.

of the acknowledging thread's delay on both brokers, and real-time priority brought it back near its idle value (Supplement S3.11). Our earlier kernel trace of every thread's wakeups [14] is consistent with the mechanism. At every threshold we read it at, around Kafka's median of S with no load, its share of longer waits stayed inside the factor of three fixed before it was read (Supplement S3.5).

4.3 The timescale the slice sets

A thread woken while another holds the core waits out what is left of the running thread's *slice*. The slice is the time the scheduler lets a thread run at a turn. A woken thread can wait up to a whole slice, because the scheduler does not preempt the running thread for it before that slice is spent. The kernel checks for a spent slice only at each *tick* h of its timer. So the wait can run up to a tick past the slice. The kernel sets the base slice s from the CPU count n, with a cap:

$$s \;=\; s_0 \times \big(1 + \lfloor \log_2 \min(n, 8) \rfloor\big), \tag{4}$$

where s_0 is 0.70 ms on the kernels we ran (Supplement S3.7). That gives 4×0.70 ms $= 2.8$ ms on our hosts' eight CPUs. The traced waits carry that timescale. They have several local maxima, which survive wider buckets. The last one sits in the bucket that contains the slice, and above it the counts collapse (Supplement S3.7). The trace also counts waits that begin when a thread is preempted, and these last about a whole slice. The thread that reads the acknowledgment is instead woken by the reply. Its own wait is the rest of a slice (Fig. 3). The registered campaign timed this wait, and we read it after its predictions were judged.

The CPU count moves the slice. In the registered campaign, the slice read back at 2, 4 and 8 CPUs as Equation 4 gives with $s_0 = 0.70$ ms. The law puts the halfway point of the share's fall inside $[s, s + h]$, from the slice to a tick past it. A free reading is a curve required only never to rise. With the slice set by hand, that curve placed the fall in the band at every slice on both brokers. The registered verdicts on these predictions are in Section 8 (Supplement S6.2).

A whole slice is 0.94–3.98 times the per-condition median end-to-end latencies we measured, which are 0.70–2.97 ms. So the slice lands on the scale of the paths now benchmarked. Together, Equations 2–4 say that the share rises with p and falls as the margin outgrows the rest of the wait, whose scale the slice sets. On our kernel, any host with at least as many CPUs as ours has a slice of 2.8 ms. So a single wait can outlast any margin shorter than that.

4.4 The same wait delays a consumer's next request

A pull consumer asks the broker for each batch of messages, as Kafka's and Redis's consumers do. One that keeps one request at a time asks again only after its thread has handled the last answer. If that thread must first wait for a core, the next request waits too. A message that reaches the broker while no request waits there stays at the broker until the next request comes. That wait at the broker enters even a latency timed by the kernel. Let W be the latency from the kernel's transmit timestamp of the message to the kernel's receive timestamp of its delivery. The message leaves the broker only when the next request arrives, so its wait at the broker is part of W. Later registered campaigns measured this on Redis and Kafka. The background load was switched on and off within each run (Supplement S8). The tail of a quantity here is its mean, in each load state, over the 1% of messages with the largest W. On Redis with one request at a time, the load raised the tail of W 2.20-fold. In each of four Kafka clients at normal priority, at least 99% of the load's rise in the tail of W was time in which the message waited at the broker for the client to ask again (Supplement S8.1). So a client thread's wait enters a latency in two places: in its clock readings and in its requests.

5 An Industry-Wide Audit

Across the industry, the tools we audited delete, replace or never measure a latency at or below zero, and almost never count it. The wait for a core enters a latency in two places (Section 4.4). Sections 5.1 to 5.3 cover the first, the clock readings: what tools do with values at or below zero. Section 5.4 covers the second, the requests: what brokers report to a consumer. Supplement S5 sorts the tools into four groups by what they time, and counts what each group does with such a value. Only the group whose two timestamps can come from two clocks can time a one-way delay across machines. The brokers they time tell a consumer at most when a message arrived, or how long it stayed at the broker in milliseconds (Section 5.4). We read 27 tools at source, ran 32 under registered predictions, measured one benchmark's filter in depth and audited the public reports made with it. We also read what 222 brokers tell a consumer and a producer about a message's time at the broker.

5.1 Tools read at source

We read 27 measurement tools at source. One program classified them, each verdict with its source line (Supplement S5). We chose them in rounds, not by a fixed rule:

first the tools that brokers' own projects ship, then every other open-source latency tool we found (Supplement S5). Of them, 24 time messages or requests. Of those, 19 cut a value at or below zero out of a figure they report, in three ways: a filter drops it, the tool's own library refuses it, or another value replaces it. All but SBK count none of what they cut. Replacement hides best, because nothing looks missing. Apache Kafka's own end-to-end tool hides a value of S below zero without computing one. It polls for each message only once the acknowledgment has returned. So a message that reached the consumer first is timed to that later moment, which replaces its arrival. It also rounds each latency down to whole milliseconds, so on a path faster than a millisecond every percentile it reports is zero [15]. Two of the cuts are edge cases. BookKeeper's benchmark drops a zero where no value below zero can arise. Kafka's tool replaces an arrival by the order of its two calls. Without them, 17 tools still cut. If the cuts made by a shipped script, or announced by the tool, are left out too, twelve do, eleven of them uncounted (Supplement S5). Of the five message tools that cut nothing, two time only to the acknowledgment, on a monotonic clock, and never meet such a value, and three keep it. We have not yet told the maintainers.

5.2 Tools run

We ran 32 benchmarking tools under predictions fixed from their sources (Supplement S5). In their figures, a wait for a core showed as time added, not as a value below zero. Of the tools we ran, 19 take both timestamps in one process. Of those, eight time a round trip and four only the acknowledgment. The rest cannot see a separate receiving process's wait for a core.

5.3 A filter on a millisecond clock

One of the cuts above is a filter that keeps only latencies above zero. It is the one cut that deletes more than the values below zero. In statistical terms it truncates the distribution at zero. What it removes is not counted, as it would be if it were censored. We measured it in depth in a broker benchmark whose end-to-end percentiles 43 public reports print, the OpenMessaging Benchmark [1]. Its filter was added in 2018 because its end-to-end latency "is calculated based on time difference of `System.currentTimeMillis()` on two different machines" and "can be negative" [16]. Its histogram rejects values below zero outright. For clocks that disagree, the filter is a sound local fix.

It has two side effects. Its clock counts whole milliseconds (Supplement S5). So its *timestamp resolution* τ is a millisecond. The timestamp resolution is the smallest step a timestamp can take. A message that arrives before the millisecond digit changes therefore computes to zero and is deleted (Fig. 4a). Nothing counts what the filter deletes. A sample survives only if a millisecond boundary falls between its two timestamps. With publish instants placed uniformly within the millisecond, that happens with probability T_{true}/τ on a path shorter than τ. Here T_{true} is the *true end-to-end latency*. It is the latency that a perfect clock, read without delay, would give. So the share of samples kept is

$$\text{share kept} \;=\; \min\!\left(1,\; \frac{T_{\mathrm{true}}}{\tau}\right). \tag{5}$$

Every sample kept on such a path reads exactly one step. So two brokers whose messages all arrive within a millisecond print the same median. They differ only in the share of samples they keep, which is never printed. The arithmetic is in Hewlett-Packard's 1970 counter note [17], but the deletion is not. That counter keeps its zero readings.

We added counters that record, by sign, every sample the filter discards, and changed nothing else. Of the 75 settings whose own summary we captured, 71 printed a median of exactly 1.0 or 2.0 ms, computed from 0.36–100% of the samples (Fig. 4b). Two replicates of one setting kept 431 and 120,434 samples, 279-fold apart, and printed the same median. The filter can also inflate a median. When it deletes the fast samples, it prints the median of the slow remainder (Supplement S4.3).

Equation 5 holds where it can be tested. A publish rate is *incommensurate* with the clock when its interval bears no ratio of small whole numbers to τ. Every configuration at such a rate keeps about the same share (Supplement S4.2).

Counted by sign, the filter hides two failures. On Kafka, the 10,913,263 samples it discarded over 214 runs hold 0 values below zero. Each of those samples computed to zero, lost to the clock's step. On Redis, the benchmark subtracts the broker's millisecond timestamp from the consumer's clock. There all 41,403 values below zero in 9 runs were exactly $-\tau$, the mark of two floored clocks less than a millisecond apart (Supplement S4.4).

Public reports. We also audited the benchmark's public reports (Supplement S5). The audit looked for the deletion's signature, a printed end-to-end median of exactly 1 ms. Of the 868 end-to-end medians in 43 reports, eight carry it, in two reports. No report states how many samples it kept. The signature is rare partly because 9 reports come from copies with a finer clock (Supplement S5).

5.4 What brokers tell a consumer and a producer

A message can wait at the broker for the next request (Section 4.4). The time it waits there enters W. Only the broker sees when that time starts and when it ends. So a tool can take that time out of W in two cases only. In the first, the broker reports how long the message waited, or the two times on its own clock. In the second, no message waits, because a request or *credit* from the consumer waits at the broker for every message. Credit is a consumer's standing permission for the broker to deliver more messages without being asked again, such as a prefetch count.

We audited which brokers allow either case. The audit covered every broker that one of the audited tools can time, the three largest clouds' own message services, and the established brokers that a wide search found. That makes 222 brokers. An *interface* is one way to consume or to publish, such as pulling and pushing. The brokers have 418 consumer interfaces and 390 producer interfaces.

Supplement S7 sums up the answers in a table, interface by interface. Of the consumer interfaces, two report the time a message waited for its request, and neither is a production broker as it ships. One is Zenoh's storage queries, through a timestamp feature that its code marks unstable. The other is Azurite, an emulator of Azure Queue Storage, to the second. 79 interfaces in 53 brokers report the message's *residence*

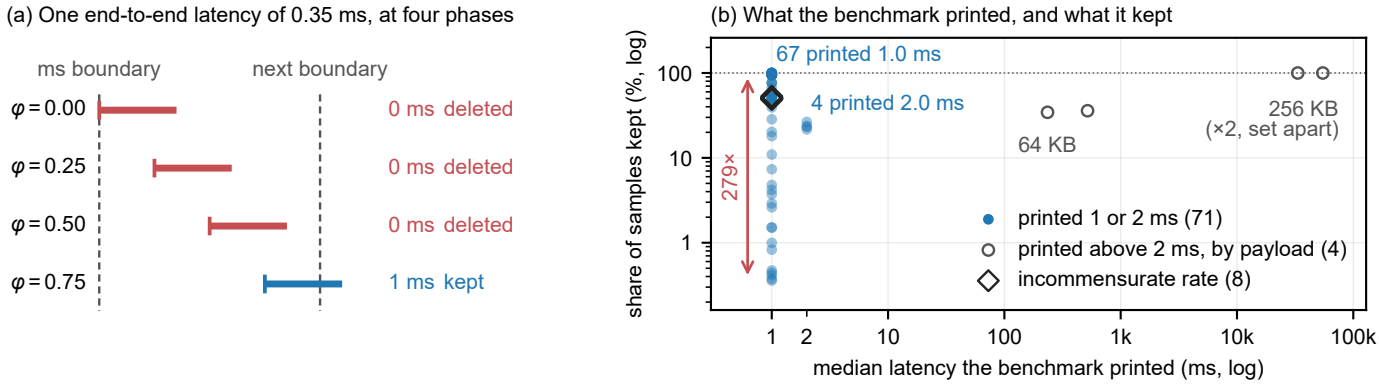


Fig. 4. **Samples that cross no millisecond boundary are deleted, and one printed median hides how much was kept.** (a) One end-to-end latency of 0.35 ms, starting at four phases φ within one millisecond: three compute to 0 ms and are deleted, and the fourth is kept, as exactly 1 ms. (b) In embedded mode, each of the 75 settings whose summary we captured, drawn at its printed median and its share of samples kept. That share spans $279\times$ among the 71 printing 1.0 or 2.0 ms. Diamonds: settings at two of the seven incommensurate rates of Section 5.3.

time. This is its whole time at the broker, from its arrival until the broker hands it on. It is the message's time waiting for its request plus the broker's own handling. So it bounds that time from above. Among the brokers that the industry's tools time, those that report it do so in milliseconds at best (Supplement S7). The other interfaces tell the consumer at best the message's arrival time (Supplement S7).

Many brokers already keep the times that a tool needs. 20 interfaces in 19 brokers record when each request arrived and keep that time inside. Kafka's fetch is among them. Such a broker could report the time with each message without measuring anything new. In 308 of the 418 consumer interfaces, a consumer can keep a request or credit waiting for every message. Kafka's fetch is not among them. Its broker holds one request at a time from each connection. So a message that arrives between an answer and the next fetch waits for that fetch.

The producer's side has the same gap. Suppose that an acknowledgment carried the time the broker received the message and the time it acknowledged it. Then a producer could take the broker's own time out of its round trip without sharing a clock with the broker. Only two producer interfaces carry both times (Supplement S7).

So for every broker that the industry's tools time, a tool can know exactly how much of W a message spent waiting at the broker for a request only where a request or credit waits for every message.

6 What to Do

A tool can time a broker's latency beneath any client and check its own reading, and a broker can report the times that only it sees. One fix ends the late reading in a Linux tool that can read its sockets' timestamps, at the rates we tested: take both of a message's timestamps from the kernel, not from the tool's threads (Fig. 5). The other steps find or shrink it where a tool still reads its own threads' clocks. Most of these steps cost little, and most were measured on our runs (Supplement S2). A general method builds on that fix (Section 6.1). On the brokers' side, a few bytes per message would tell every tool how long each message waited at the broker (Section 6.3).

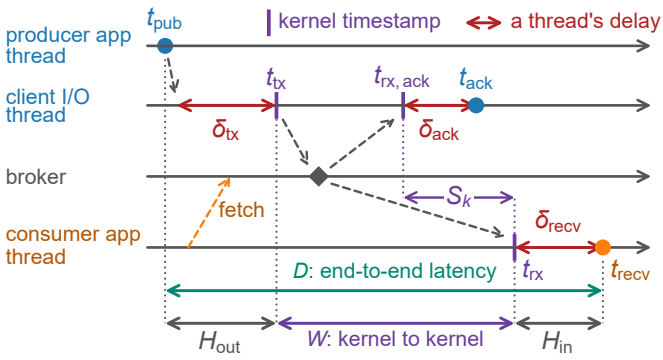


Fig. 5. **Timed between the kernel's timestamps, a message's latency holds no client thread's wait at fifty messages a second.** Kafka's path. The client's I/O thread writes the message after its delay δ_{tx}, and the kernel timestamps it as it leaves, t_{tx}. The consumer's request is already at the broker, which answers with the message. The acknowledgment and the message reach the client machine at $t_{rx,ack}$ and t_{rx}, before the delays δ_{ack} and δ_{recv}. $W = t_{rx} - t_{tx}$ holds the network and broker. As drawn, each client thread's wait on the message's path falls in H_{out} or H_{in}, and $S_k = t_{rx} - t_{rx,ack}$ holds no wait.

6.1 The tools' side: a general method

Let the kernel timestamp each message. A tool reads each of a message's timestamps in one of its threads, which may first wait for a core. The kernel timestamps each packet as it leaves and enters a machine, the transmit and receive timestamps, without waiting for any program's thread. The latency W, from a message's transmit timestamp to its receive timestamp, therefore holds the network and the broker but no client thread's wait, whenever each socket read drains one packet and a consumer request already waits at the broker. No wait is lost: H_{out} runs from the publish call to the transmit timestamp and H_{in} runs from the receive timestamp to the consumer's reading, so $D = H_{out} + W + H_{in}$ for every message (Fig. 5). This is network measurement's rule: delay is defined between times on the wire, not the host times a program reads [18], and the one-way delay standard bounds each host's share by terms that H_{out} and H_{in} measure [7]. A tool takes these timestamps from its own sockets where its language allows, or else from a packet capture on the client machine, with no change to the client.

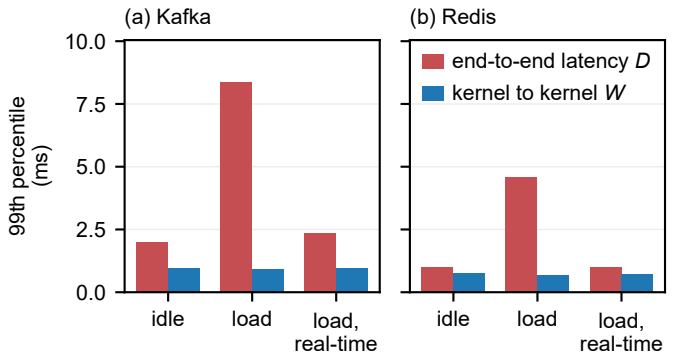


Fig. 6. **Timed by the kernel, a message's latency holds level under load, while the tail that benchmarks report grows 4.2–4.7-fold.** The 99th percentile of D, timed by the clients' threads, and of W, read on their own sockets, at fifty messages a second: on an idle client machine, under 88% background load at normal priority, and under that load at real-time priority, which takes most of D's rise back. Each bar is a setup's median over its runs (Supplement S3.10).

It names each message by the broker's own id, such as a Kafka offset or a Redis entry id.

All five predictions of the fix's first test, fixed before any run, held. Over 48 runs on both brokers, timed from packet captures, the median of W moved at most 7.9% and its 99th percentile 7.3% across load and priority, inside the bands fixed before the runs. Their run-level intervals reach beyond the bands, so the bands hold for the point estimates only (Supplement S3.9). In the same runs, load at normal priority raised the 99th percentile of D 4.3–4.6-fold. A delay added between broker and consumer raised W by about as much as the delay (Supplement S3.11). W never reached zero, so a filter that keeps only latencies above zero deletes nothing from it.

Read those timestamps on the tool's own sockets. A tool need not capture packets. On Linux it reads these timestamps through the `SO_TIMESTAMPING` *socket option*, a setting a program makes on a socket, in four steps [19], whose settings Supplement S3.10 gives in full.

1) On each connection to the broker, before its first write, turn on the kernel's software timestamps for transmit and receive.
2) Read each write's transmit timestamp from the socket's *error queue*, where the kernel leaves it. Read the error queue at every write and read, since while it holds a timestamp a client that polls the socket spins.
3) Take the receive timestamp the kernel returns with each read, that of the last packet the read consumed.
4) Find each message in the bytes by an identifier the benchmark puts in each payload; this needs a connection without compression or encryption. Time it from the transmit timestamp of the write that carried it to the receive timestamp of the read that delivered it. Report that latency beside D, and count any message left without a timestamp.

Lancet reads a request-reply service's timestamps this way [20], Fathom splits a fleet's call latency into host and network parts with kernel instrumentation [21], and Zuo et al. split a host's network-stack latency at kernel timestamps [22]. We read these timestamps beneath unmodified broker clients, on the one-way path from producer to consumer. Our Python module takes these steps for both brokers (`scripts/so_timestamps.py`). In 48 more runs it came within 3.1 μs of the captures' latency at the median. Both read the kernel's receive timestamp, so this shows the module matches timestamps to messages. The network card's own timestamps bound how far the kernel's timestamps lie from the wire (Supplement S3.9). Under load at normal priority, the tail of D again grew 4.2–4.7-fold, while the latency the module timed held level (Fig. 6). Against the capture runs, taking the timestamps moved the clients' own time, $D - W$, and the 99th percentile of D by less than a fifth of a millisecond either way, the drift between two days included (Supplement S3.10). The sockets agreed with the captures up to $2{,}000$ messages a second (Supplement S3.11).

What a tool must handle. A read that drains several packets times the earlier ones late, by their wait in the socket, which is the consumer's wait. A pull consumer, which asks the broker for each batch as Kafka's and Redis's do, adds another: a message the broker stores while no request is outstanding waits for the next one, which the consumer's thread issues after its own wait, inside W. A tool can flag such messages, those delivered in answer to a request whose transmit timestamp follows the message's own; Supplement S3.11 counts them at each rate. Where receive processing runs in a kernel thread, the receive timestamp waits for a core too (Supplement S3.9). Under threaded network polling, for example, a kernel thread polls the network card. The kernel's deferred interrupt work, the softirq, also moves to kernel threads, `ksoftirqd`, when it falls behind. Where a driver gives no software transmit timestamp, as on our cloud's accelerated network, the transmit timestamp left to a tool is taken before the packet scheduler, so any queueing there falls inside W (Supplement S3.10). With producer and consumer on two machines, W spans two clocks and includes their offset, so they must be synchronized well below the latency. A design in which the consumer acknowledges back to the producer times two legs on one clock, not the one leg wanted. Recovering the two clocks' offset from logs instead is itself synchronization. Java's standard sockets expose neither the option nor the error queue, so a Java tool needs a native call (Supplement S3.10). A kernel-bypass stack has no kernel timestamps at all. A tool that cannot yet take them needs the checks of Section 6.2.

Check whether a request was waiting. The same timestamps can settle, for each message, whether a request or credit was waiting at the broker on its arrival, wherever a bound on the network's delays allows (Supplement S8.2). A tool reports the share of messages that found none beside the latency. Plan version 47 tested this reading. In each run that it judged, the reading never said that a message found none when the broker's own capture showed one waiting. In plan version 49, in each run whose own checks held, the clients' own timestamps showed a waiting request or credit for every message. That held on Redis, on NATS JetStream and on RabbitMQ (Supplement S8).

Report three latencies. F is the latency to a receiver as ready as the protocol allows: what the broker and the

network deliver when a request already waits, or, on Kafka, when the client asks again at once. W is the latency to the consumer's kernel, and D the latency to the application. $W - F$ is the client's part, its wait to ask again. $D - W$ is the time spent in the two clients, $H_{\text{out}} + H_{\text{in}}$, their threads' waits for a core included.

Keep a request or credit waiting, by protocol. Where the protocol allows it, the consumer should ask ahead: it keeps enough requests or credit at the broker that each arriving message finds one. Then no message waits for the next request, so W holds none of the client threads' waits. Plan version 49 tested this (Supplement S8). On Redis, NATS JetStream and RabbitMQ, the tail of W then held still under the load, while D, over its own largest 1% of messages, grew 3.3- to 17.9-fold. Kafka's broker holds one fetch at a time from each connection. There the remedy is a fetch thread that does nothing else, raised to real-time priority. In librdkafka, the broker thread is such a thread. With that thread raised, the client's part, over its own largest 1%, rose under the load by at most 0.33% of its rise at normal priority, on both pairs (Supplement S8.2). The fetch's round trip stays in W, and it is real latency. All 13 predictions of plan version 49 held (Supplement S8). They were chosen after reading plan version 47's runs of the same setups, and every one held on those runs, so the new runs are their only test. Plan version 47's own test of the held tail of W was not confirmed. The tail held still, but the load raised D less than that test required (Supplement S8.2).

6.2 Checks for benchmark authors and readers

Check every run for values below zero. The *check for values below zero* rejects a run when more than one percent of its messages carry a value below zero in any latency component it records (S, the publish delay or the processing-time latency), or when the median of any of them is below zero. It rejected 51.9% of the cloud runs behind our results (Supplement S2). It sees only the worst cases: the publish latency exceeds half the end-to-end latency on 77.6% of messages, but the check is triggered only by the 8.79% on which it exceeds the whole, and a latency that load inflates but leaves positive passes the check by construction. The check also assumes a broker that acknowledges no later than it delivers. On one that delivers first, as Redis often does, S can fall below zero without a late reading. So there the check should read only the differences timed from the publish call. The check therefore needs the repair that follows.

Time from the publish call, or add the publish latency back. The repair is Equation 1: $D = S + A$ for every message, and A is read on the producer's side of the same clock, so a benchmark that records its publish latency recovers each message's end-to-end latency exactly. Error arises only where a benchmark keeps summaries. Over groups of runs, the median of S plus the median of A misses the median of D by a median of 0.4% on Kafka and 14.6% on Redis (Supplement S2.1). The check's verdict made no detectable difference to that error on either broker. The publish latency does not grow as leg 2 lengthens. On one path, the relative error of S as a latency, A/D, therefore falls as $1/D$. Below $D = 0.68$ ms the publish latency exceeds D itself. From one condition to the next, though, Kafka's A grows with D, so its groups lie above that curve (Supplement S2.1). Timing from the publish call removes the values below zero, but not the consumer's own wait for a core, which lengthens every latency, unseen by any check. In our runs it added 0.41–0.94 ms to the mean of D (Supplement S3.8), a wait that W leaves out.

Give the threads that read the clock a core. Real-time priority cut the share of S below zero in every matched pair. At real-time priority some values of S below zero remain (Supplement S3.1), so the check stays. A real-time or spinning thread also changes the system it measures.

A better-synchronized clock repairs neither the late reading nor the deletion: the values of S below zero arise on one clock, and the deletion from the clock's step. A finer clock confines the deletion to faster paths but leaves the filter and every value of S below zero in place. The remedies are therefore ordered: record at native resolution, count what is discarded and randomize the publish instants, and only then improve the clocks' synchronization.

6.3 The brokers' side

A broker sees when a message starts to wait for its request and when the request arrives, so it can measure that time exactly. It should report, with each message it delivers, how long the message waited: from the moment the message could be delivered to the arrival of the request or credit that took it. It should take both readings on its own monotonic clock, on the node that serves the request. Written as a variable-length integer of microseconds, that time takes at most three bytes below $2{,}097{,}152\,\mu$s (Supplement S7.1). With that time reported, every tool can take each message's wait for its request out of W, whatever the protocol.

Where the time goes matters. A broker such as Kafka keeps its batches with a checksum and passes them from its files to the network unchanged, so a time per delivery belongs in the envelope of the response, not inside the stored batch. It should be a numbered field, not a named header. Names cost bytes, and different brokers give one time different names (Supplement S7.1). A broker should add the field only when a consumer asks for it, since an unrequested field can break older clients (Supplement S7.1).

For the producer, each acknowledgment should carry two times: when the broker received the publish and when it acknowledged it. The Network Time Protocol (NTP) gives its clients a server's two times in the same way, and two of the producer interfaces we audited already do (Section 5.4).

7 Related Work

Scheduling delay. That a ready thread can wait for a core is well known [14]. Behind another thread's slice, the wait adds milliseconds to a server's tail latency, and real-time priority removes it [23]. Co-located work lengthens it [24], a longer slice extends it [25], and real-time Linux measures it as noise [26]. In Linux's network stack, the time a reader woken by arriving data waits to run is the main source of tail latency [22], and the limits of the timers that benchmarks read are known too [27]. Load testers of microsecond-scale services correct for the *client bias* that their own queueing and scheduling add [28], [20]. Villain et al. found that an

outgoing socket timestamp “does not respect causality” under every stress pattern, including none [29]. None gives the consequence for a difference read by two processes: its share below zero is set by p, the probability that the thread is waiting for a core, and by the margin, on the timescale of the slice.

Clocks. Across hosts, values below zero are read as clocks that disagree, and on one host as a thread preempted after it read the clock [30], but neither reading measures the share or its laws. Sharma et al. read negative timing spans as clock skew, since “queueing alone cannot produce negative timing spans” [31]. For the backlog they model we agree, but a timestamping thread waits in a second queue, the kernel’s run queue. Van Dongen and Van den Poel, seeing latencies below zero on up to a tenth of their events, took both timestamps from one broker [32], the one-clock rule reached by another route. Network measurement defines delay between wire times [7], and Lancet and Fathom read such times through the kernel [20], [21]. We read them beneath unmodified broker clients.

Resolution and discarded samples. The arithmetic of the share kept and its cure, phase jitter, are in a 1970 counter note [17], and a clock coarser than the intervals it times can still estimate their mean [33]. IP measurement advises randomizing probe times [18], [34], since a periodic schedule can synchronize with periodic behavior [35], here the clock’s step. Coordinated omission [36] is the deletion’s mirror image: it fails to record samples that would have been slow. Network measurement reports the share of traces it rejects [37] and keeps a packet timestamped in the future [38], and rigorous benchmark reporting asks that removed outliers be counted [39], [40]. The tools we audited remove samples before any analyst sees them and count none, a case that neither that literature nor the principles for reproducible performance evaluation in the cloud [41] address. Stream-processing benchmarks compute latency from Kafka’s millisecond timestamps of a record and of its result [42], so the same arithmetic applies wherever their latencies fall below a millisecond. We add these checks, applied where a benchmark reports a latency.

8 Limitations

That scheduling makes both brokers’ readings late is now observed message by message (Section 4.2). One clock excludes clock skew, and the controlled experiments exclude utilization and stress. Real-time priority excludes the interrupt noise that reaches every scheduling class, while timer slack, which differs between classes, cannot delay a wakeup on arriving data. Clocks that disagree across virtual CPUs are excluded too. On the Oracle machines a clock whose read costs at most 90 ns can only be one the kernel uses where the CPUs’ clocks agree. The Azure x86 machines read the processor’s timestamp counter, which the kernel uses as its clock only where the counters agree (Supplement S3.6). Since D never fell below zero, no offset can exceed its smallest value, 454 times smaller than the deepest value of S. Both testbeds are virtual, so *steal time*, the time a virtual processor is ready but waits while the hypervisor runs another machine, is bounded, not excluded: real-time priority, a setting inside our virtual machine, cannot shorten it, yet it cut the share. On the Oracle machines, our Kafka and Redis clients ran side by side, so each broker’s numbers there include the other’s clients in the load (Supplement S1.1). The rival we had left open, a broker that delivers before it acknowledges, is real on Redis and rare on Kafka (Section 4.1). By our registered rule, Redis’s values of S below zero with the delivery first count as its reply order, though the readings’ delays still decide which delivery-first messages fall below zero. That order lets Redis’s real-time share stay above zero: across the matched pairs it stayed near two percent at every load, while the normal-priority share rose by about two-fifths (Supplement S3.1). Our CPython timestamps also wait on the interpreter lock, and Kafka’s Java client, which has none, shows values below zero too, at a third to a quarter of Python’s share (Supplement S6.2), so the effect needs no lock. With one publish worker instead of sixteen, Redis’s real-time share fell more than thirtyfold with no load, so there the lock the workers share sets most of it, though not under load (Supplement S3.11). We have not tested why placement moves the two brokers’ shares in opposite directions, nor traced the cause of Redis’s mode near $-200\,\mu$s in Fig. 2; on a second pair, a processor waking from idle did not cause it (Supplement S3.11).

Equation 3 describes how the share moves between conditions. Inside one, the share rises with D in 24 of 56 testable conditions, because a long delivery comes with a long publish latency (Supplement S3.4). A registered prediction that priority would cut p at least tenfold failed: by the scheduler’s counters, which average away the tail, it fell only about twofold, so the equation’s factorization is untested between conditions. In the registered campaign, none of three predictions of the share from recorded waits held within the registered factor of 1.5 on every pair and broker. The share of readings below zero whose thread had mostly been waiting reached the registered 90% on four of twelve combinations. The law’s fitted shape placed the fall in the slice’s band less often than the free reading, and at no CPU count on Redis. No prediction of where the share falls at a hand-set slice passed both readings, and those of how quickly it falls held on one broker at most (Supplement S6.2). Nor, with the margin read on the kernel’s timestamps, did the share fall from each fifth of the margin to the next, as registered. Turning off the scheduler’s run-to-parity rule, which lets a running thread keep its core until its slice is spent, moved neither the share nor the acknowledging thread’s delay, against a registered cut to two-thirds, so the rule alone does not hold a woken thread off the core (Supplement S3.11). The kernel trace pools both brokers’ threads and is not free: under it Kafka’s real-time runs read no value below zero, against 15 in $2{,}985$ untraced. The traced real-time configurations therefore cannot test the trace check, though under the trace the normal-priority share did not move (Supplement S3.5). The registered campaign’s recording raised the share it recorded up to 4.33-fold, so each prediction is held against its run’s own share. The base slice on the Oracle hosts is derived, not read off them; the Azure hosts read it back. The constants are those of one EEVDF-era kernel, before Linux 6.17 limited the run-to-parity rule, a change for threads that set their own slices. The distribution and the laws were measured with paced producers at about one message a second each, and fifty

messages a second in the registered campaign, where the threads that read the clock sleep between messages and must be woken. At 500 messages a second on an idle host, two of eight runs had most values of S below zero for the whole run. The laws were not tested above fifty messages a second (Supplement S3.11). The late request (Section 4.4) was shown on one x86 pair, for Redis and Kafka only. On NATS and RabbitMQ, the consumers we tested fell behind $2{,}000$ messages a second, so the test could not show it (Supplement S8.1). The rule that leaves out messages delayed by a testbed fault was chosen after the Oracle testbed was gone (Supplement S1.5).

The fix was tested from packet captures and, through a Python module, on the clients' own sockets. Those tests used one pair of machines and one kernel, with one partition, background load on the client machine alone, and the producer and the consumer on one machine. Its stillness under load was judged at fifty messages a second only. On a second pair, under load, Kafka's W held within 6.3% and Redis's rose 26%. At $2{,}000$ messages a second, with nearly every message flagged, a pull consumer's wait entered W. On that virtual network a departing packet's timestamp came only from before the packet scheduler, and the clients used neither encryption nor compression. We tested no designs that timestamp in hardware or pin a spinning thread to its own core (*busy-polling*), which keep the waiting state out of the latency they time. The general method was tested at $2{,}000$ messages a second on one x86 and one Neoverse N2 pair, with the load only on the client machine. Plan version 47 confirmed eight of its 18 predictions, did not confirm nine and could not judge one. RabbitMQ's credit was shown waiting only at a bound of 50 ms (Supplement S8.2).

The deletion is demonstrated in one benchmark, in its embedded mode, and reproduced in an independent program of ours; the benchmark's distributed mode, run on the Azure machines, dropped values below zero under a moved clock, uncounted (Supplement S4.5). In the registered change of message size (Supplement S4.3), the 32 KB configuration pinned at a share kept just above the predicted two-thirds. The audit found the deletion's signature in few public results, so we claim it for those and our own runs only. We chose the tools in rounds, not by sampling, classified the 27 tools by reading them, and ran the 32 tools on one machine type, at one publish rate where a tool lets it be set. The broker audit ran no broker. It read sources only, and it left the products below its scope rule unchecked (Supplement S7). The brokers' side of Section 6.3 is a proposal that we have not built. How much a deployment loses depends on its end-to-end latency and publish rate, so we recommend reporting the share of samples kept rather than calling any reported number wrong.

9 Conclusion

A latency benchmark measures the system only while the delay before each reading is short beside the latency it measures. On one machine's clock, the moment a producer reads a broker's acknowledgment and the moment a consumer reads the message can differ by milliseconds, in either order. A thread waits for a core, for up to a scheduler's slice and a tick, before it reads the clock. That wait can also postpone a pull consumer's next request and hold the message at the broker meanwhile. Across the industry, values at or below zero are deleted, replaced or never measured. Where the clock counts whole milliseconds, discarding every value at or below zero also discards the fastest messages, and no report we audited says how many samples it kept. None of the brokers those tools time reports how long a message waited for its request, or a residence time finer than a millisecond. On Linux a tool can time each message between the kernel's timestamps, read on its own sockets, and report that latency beside the end-to-end latency. At fifty messages a second, the latency between the kernel's timestamps held no client thread's wait. The same timestamps can show whether a message found a request or credit ready at the broker, and a broker could add, in a few bytes, how long each message waited for one. Before acting on a broker latency below a few milliseconds, ask what it was timed from, how many samples the tool kept, and how many it discarded, by sign.

Acknowledgment

The authors thank N. Herbst, for the load-generation point that became one of the checks above, for pointing us to the timer study cited in Section 7, and for a reading that reset the abstract.

In accordance with IEEE policy, Anthropic's Claude assisted with code development and verification, literature search, and the drafting and editing of text in every section, under the authors' direction and review. Experimental design, scientific claims, and interpretation are the authors'.

Artifact Availability

Code, analysis and manuscript are archived at 10.5281/zenodo.21650031, the measurement dataset at 10.5281/zenodo.21650064; both resolve to the current version, `v4.0.0`, whose files are restricted pending publication. Every number here is recomputed from the committed data by the archived code at build time, and the test suite fails if the text and the data disagree.

References

[1] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/. Accessed: Aug. 19, 2026.

[2] Redis, "Redis streams: Latency tests results," https://redis.io/docs/latest/develop/data-types/streams/, 2026, accessed: Oct. 15, 2026.

[3] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan, "Attack of the killer microseconds," *Commun. ACM*, vol. 60, no. 4, pp. 48–54, Apr. 2017.

[4] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013.

[5] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Mar. 2009, pp. 265–276.

[6] M. Baker, "1,500 scientists lift the lid on reproducibility," *Nature*, vol. 533, no. 7604, pp. 452–454, May 2016.

[7] G. Almes, S. Kalidindi, M. Zekauskas, and A. Morton, Ed., "A one-way delay metric for IP performance metrics (IPPM)," Internet Engineering Task Force, RFC 7679, Jan. 2016, STD 81.

[8] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proc. IEEE 34th Int. Conf. Data Engineering (ICDE)*, Apr. 2018, pp. 1507–1518.

[9] I. Stoica, H. Abdel-Wahab, K. Jeffay, S. K. Baruah, J. E. Gehrke, and C. G. Plaxton, "A proportional share resource allocation algorithm for real-time, time-shared systems," in *Proc. 17th IEEE Real-Time Systems Symp. (RTSS)*, Dec. 1996, pp. 288–299.

[10] StatsBomb, "Statsbomb open data," *GitHub Repository*, 2023, accessed: Oct. 15, 2026. [Online]. Available: https://github.com/statsbomb/open-data

[11] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. Natl. Acad. Sci. USA*, vol. 115, no. 11, pp. 2600–2606, Mar. 2018.

[12] OpenMessaging Benchmark contributors, "Can the average publish latency be larger than end-to-end latency? (issue #216)," https://github.com/openmessaging/benchmark/issues/216, 2021, accessed: Oct. 15, 2026.

[13] D. Bristot de Oliveira, D. Casini, J. Lelli, and T. Cucinotta, "Timerlat: Real-time Linux scheduling latency measurements, tracing, and analysis," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2608–2620, Aug. 2025.

[14] B. Gregg, "Linux bcc/BPF run queue (scheduler) latency," https://www.brendangregg.com/blog/2016-10-08/linux-bcc-runqlat.html, Oct. 2016, accessed: Sep. 13, 2026.

[15] Apache Software Foundation, "Apache Kafka tools — `EndToEndLatency.java` and `ProducerPerformance.java`," Source repository, 2026, https://github.com/apache/kafka. Accessed: Aug. 31, 2026.

[16] S. Guo, "Avoid adding negative metric values (pull request #56)," OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, https://github.com/openmessaging/benchmark/pull/56. Accessed: Aug. 18, 2026.

[17] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766.

[18] V. Paxson, G. Almes, J. Mahdavi, and M. Mathis, "Framework for IP performance metrics," Internet Engineering Task Force, RFC 2330, May 1998.

[19] Linux Kernel Documentation, "Timestamping," https://docs.kernel.org/networking/timestamping.html, 2026, accessed: Oct. 15, 2026.

[20] M. Kogias, S. Mallon, and E. Bugnion, "Lancet: A self-correcting latency measuring tool," in *Proc. USENIX Annual Technical Conf. (ATC)*, Jul. 2019, pp. 881–896.

[21] M. A. Qureshi, J. Yan, Y. Cheng, S. Hassas Yeganeh, Y. Seung, N. Cardwell, W. De Bruijn, V. Jacobson, J. Kaur, D. Wetherall, and A. Vahdat, "Fathom: Understanding datacenter application network performance," in *Proc. ACM SIGCOMM 2023 Conf.*, Sep. 2023, pp. 394–405.

[22] T. Zuo, J. Hwang, A. Tang, R. Agarwal, and Q. Cai, "Understanding host network stack latency," in *Proc. ACM SIGCOMM 2026 Conf.*, Aug. 2026, pp. 1127–1140.

[23] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, "Tales of the tail: Hardware, OS, and application-level sources of tail latency," in *Proc. ACM SoCC*. ACM, Nov. 2014, pp. 1–14.

[24] J. Leverich and C. Kozyrakis, "Reconciling high server utilization and sub-millisecond quality-of-service," in *Proc. 9th European Conf. Computer Systems (EuroSys)*, Apr. 2014, pp. 1–14.

[25] Y. Jia, K. Tian, Y. You, Y. Chen, and K. Chen, "Skyloft: A general high-efficient scheduling framework in user space," in *Proc. 30th ACM Symp. Operating Systems Principles (SOSP)*, Nov. 2024, pp. 265–279.

[26] D. Bristot de Oliveira, D. Casini, and T. Cucinotta, "Operating system noise in the Linux kernel," *IEEE Trans. Comput.*, vol. 72, no. 1, pp. 196–207, Jan. 2023.

[27] M. Kuperberg, M. Krogmann, and R. H. Reussner, "Metric-based selection of timer methods for accurate measurements," in *Proc. 2nd ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, Mar. 2011, pp. 151–156.

[28] Y. Zhang, D. Meisner, J. Mars, and L. Tang, "Treadmill: Attributing the source of tail latency through precise load testing and statistical inference," in *Proc. 43rd ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, Jun. 2016, pp. 456–468.

[29] B. Villain, M. Davis, J. Ridoux, D. Veitch, and N. Normand, "Probing the latencies of software timestamping," in *Proc. IEEE Int. Symp. Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, Sep. 2012, pp. 1–6.

[30] P. E. McKenney, "Is parallel programming hard, and, if so, what can you do about it?" arXiv preprint arXiv:1701.00854v6, Jun. 2023, release v2023.06.11a, App. A.3–A.4.

[31] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, "Time, causality, and observability failures in distributed AI inference systems," *arXiv preprint arXiv:2604.21361*, Apr. 2026, Open Compute Project, Unified Intelligent Infrastructure workstream.

[32] G. Van Dongen and D. Van den Poel, "Evaluation of stream processing frameworks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1845–1858, Aug. 2020.

[33] P. B. Danzig and S. Melvin, "High resolution timing with low resolution clocks and a microsecond resolution timer for sun workstations," *ACM SIGOPS Operating Systems Review*, vol. 24, no. 1, pp. 23–26, Jan. 1990.

[34] F. Baccelli, S. Machiraju, D. Veitch, and J. C. Bolot, "The role of PASTA in network measurement," in *Proc. ACM SIGCOMM*, Sep. 2006, pp. 231–242.

[35] V. Raisanen, G. Grotefeld, and A. Morton, "Network performance measurement with periodic streams," Internet Engineering Task Force, RFC 3432, Nov. 2002.

[36] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop 2015, St. Louis, MO, USA. https://www.youtube.com/watch?v=lJ8ydIuPFeU.

[37] V. Paxson, "On calibrating measurements of packet transit times," in *Proc. ACM SIGMETRICS*. ACM, Jun. 1998, pp. 11–21.

[38] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, "A one-way active measurement protocol (OWAMP)," Internet Engineering Task Force, RFC 4656, Sep. 2006.

[39] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, Oct. 2007, pp. 57–76.

[40] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, Nov. 2015, pp. 73:1–73:12.

[41] A. V. Papadopoulos, L. Versluis, A. Bauer, N. Herbst, J. von Kistowski, A. Ali-Eldin, C. L. Abad, J. N. Amaral, P. Tůma, and A. Iosup, "Methodological principles for reproducible performance evaluation in cloud computing," *IEEE Trans. Softw. Eng.*, vol. 47, no. 8, pp. 1528–1543, Aug. 2021.

[42] S. Henning, A. Vogel, M. Leichtfried, O. Ertl, and R. Rabiser, "ShuffleBench: A benchmark for large-scale data shuffling operations with distributed stream processing frameworks," in *Proc. 15th ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, May 2024, pp. 2–13.

Gustavo Pedro Ricou received the B.Sc. degree in physics from the Universidade do Porto, Portugal, and specialized in nuclear fusion science at the Universität Stuttgart, Germany, and the Université de Lorraine, France. He holds an M.Sc. in software design with cloud-native computing from the Technological University of the Shannon, with first-class honors, and is currently pursuing an M.Sc. in statistics and data science at Trinity College Dublin, Ireland. He is a Software Engineer with BoscaSports. His research concerns measurement validity — what a reported number is really evidence of — in computer systems, environmental measurement and sports analytics, using pre-registration and physical-consistency auditing.

Romaric Duvignau received the Ph.D. degree in computer science from the University of Bordeaux, France, in 2015, for work on the maintenance and simulation of dynamic random graphs. He is an Associate Professor and Head of the Network and Systems unit at Chalmers University of Technology, Gothenburg, Sweden. His research concerns distributed and data-driven algorithms, the monitoring of large-scale distributed systems, and stream analytics for cyber-physical systems.